

## Case Description

LangChain is an advanced framework for developing language model-powered applications. One famous use case is creating customer support or Q&A chatbots. In this Create a Q&A Chatbot with LangChain Project, discover how to create a chatbot using Python and LangChain for the Introduction to Tableau course by Ned Krastev.

To achieve this goal, you'll employ Retrieval Augmented Generation (RAG)—a technique that has become increasingly popular for building AI-powered chatbots. This LangChain tutorial encompasses three stages: indexing, retrieval, and augmented generation.

First, we index by dividing the Introduction to Tableau course transcript into shorter sections with LangChain's Markdown header and token splitters. Next, we convert these sections into numerical vectors through embedding and store these embeddings in a Chroma vector store.

In the second stage, a retriever extracts the portion of the course that is most valuable for addressing a student's question.

In the final stage, the chat model uses the retrieved text and students' questions to generate an "augmented" response, demonstrating how to make a chatbot that provides accurate and contextual answers.

To implement the chain (representing the Q&A chatbot), we'll employ the LangChain Expression Language (LCEL) protocol. It allows connecting Runnable LangChain components so that the output of one serves as the input to the next component.

The student's question is inputted into a prompt template. Filling in the template creates a complete prompt to feed to a chat model. The response from the chat model is then fed to an output parser, ensuring we return the result to the student as a string.

The Create a Q&A Chatbot with LangChain Project will test your skills in:

- Utilizing LangChain's text splitters
- Managing chat messages and chat prompt templates
- Applying LLMs and constructing LCEL chains for course transcription correction
- Generating embeddings for storage in a vector store
- Constructing RunnableLambdas from regular Python functions and lambda functions
- Creating LCEL chains for a Q&A chatbot

So, ready to learn how to build a chatbot using LangChain and OpenAI? Let's begin this exciting chatbot project!

## Project requirements

For this Create a Q&A Chatbot with LangChain Project, you'll need a valid OpenAI API key.

**Note:** This project necessitates using OpenAI's embedding and language models. Accordingly, *monitor* your token consumption and the associated charges.

You'll also need the Jupyter Notebook up and running.

You're advised to create and work in a new environment specifically designed for this exercise, with a version of Python  $<4.0$ ,  $\geq 3.9$ . The following installations are required for this LangChain project:

- `langchain v0.3.3`

**Note:** `pip install langchain==0.3.3` will also install `langchain-core` and `langchain-text-splitters`—both required for the project.

- `langchain-chroma v0.1.4`
- `langchain-community v0.3.2`
- `langchain-openai v0.2.2`
- `pypdf v5.0.1`
- `python-dotenv v1.0.1`

You can execute the chatbot project with different package versions. Be mindful, however, of package incompatibilities and differences in the LangChain syntax.

Since LangChain is subject to continuous updates, you might encounter minor syntax differences compared to the Build Chat Applications with OpenAI and LangChain course. If this occurs, consult the LangChain API Reference page for current classes and methods.

## Project files

The `Introduction_to_Tableau.pdf` file contains a transcript of the Introduction to Tableau course. The `LangChain_Project.ipynb` Jupyter Notebook includes a skeleton of the code used throughout the project. Download these files to create a chatbot step by step in this LangChain project.

## Load the Course Transcript

In this Create a Q&A Chatbot with LangChain Project, you'll implement a chatbot for the Introduction to Tableau course by Ned Krastev. To achieve this goal, you'll employ Retrieval Augmented Generation (RAG) encompassing three stages: indexing, retrieval, and augmented generation.

First, we index by dividing the Introduction to Tableau course transcript into shorter sections with LangChain's Markdown header and token splitters. We convert these sections into numerical vectors through embedding and store these embeddings in a Chroma vector store.

In the second stage, a retriever extracts the portion of the course that is most valuable for addressing a student's question.

In the final stage, the chat model uses the retrieved text and students' questions to generate an "augmented" response, demonstrating how to make a chatbot that provides accurate and contextual answers.

Ready to learn how to build a chatbot using LangChain and OpenAI? Let's begin!

As with any project you undertake, best practice suggests you create a new virtual environment specifically tailored to this task—with the Python version and package set described in the Project Requirements section at the front page of the project.

Once you have this set up, open the `LangChain_Project.ipynb` file using your new environment. In the notebook, you'll find a skeleton of the code. Your first task is to execute the cell under "Set the OpenAI API Key as an Environment Variable."

The following line of code:

```
%load_ext dotenv
```

```
%dotenv
```

loads the `dotenv` extension and sets the content of a `.env` file as an environment variable. This implies that you have a `.env` file already prepared with your OpenAI API key stored in the format:

```
OPENAI_API_KEY="..."
```

For convenience, keep the `.env` file in the same folder as the `LangChain_Project.ipynb` file.

Next, execute the cell under "Import the Libraries." If any libraries are missing, `pip`-install them in your project environment, restart the notebook's kernel, and run the cell anew.

Once you've successfully executed these first two cells, you're ready to continue with the fun part!

1. Prepare a loader object for the `Introduction_to_Tableau.pdf` file. Call it `loader_pdf`.
2. Load the content of the file to a variable called `docs_list`.
3. Before advancing with the code, open the PDF file and study its content. The course transcript is organized into lectures, with the names of the lectures preceded by two hash signs. The lectures are further organized into sections, with the names of the sections indicated by a single hash sign. These marks will later help split the transcript into sections and lectures using LangChain's `MarkdownHeaderTextSplitter`.

Since the `MarkdownHeaderTextSplitter` class accepts a single string as an argument, our task is to concatenate the contents of the `Document` objects in the `docs_list` into such a string. Store the string (containing the entire course transcript) in a variable called `string_list_concat`.

## Split the Course Transcript with `MarkdownHeaderTextSplitter`

Next up in our game plan is splitting the text using LangChain's `MarkdownHeaderTextSplitter` class. Doing so will create a list of `Document`

objects, each corresponding to a single lecture, with the section and lecture names stored in the metadata.

All right, let's execute this plan.

1. Create an instance of the `MarkdownHeaderTextSplitter` class and call it `md_splitter`. Ensure that Header 1 titles are stored as "Section Title" and Header 2 titles correspond to "Course Title."
2. Use the object to split the `string_list_concat` variable. Store the result in a new one called `docs_list_md_split`.

## Create a Chain to Correct the Course Transcript

All right! We now have the transcript split into documents, with each document corresponding to a separate lecture from the course. But if you inspect the content of the documents closely, you'll find that the transcript is not without flaws—the punctuation seems incorrect, the paragraphs are not split well, and some of the words are plain wrong.

So, to implement a high-quality final product, we must transform the page contents so that the text is readable. Of course, doing that by hand would be very time-consuming. So, let's employ an LLM to do that for us.

The approach would be to create a simple LCEL chain consisting of a chat prompt template (where we'll pass the lecture scripts one at a time), a chat model, and a string output parser. We'll then invoke the chain to pass all lecture texts in parallel.

Let's execute this plan!

1. Extract the lecture texts from all `Document` objects in `docs_list_md_split`. Store them as a list of strings (`string_list_split`) so an individual string stores a particular lecture.
2. Study the `PROMPT_FORMATTING_S` and `PROMPT_TEMPLATE_FORMATTING_H` strings defined in the Jupyter Notebook file (the letters "S" and "H" at the end of the variable names refer to "system" and "human," respectively.) Locate the `lecture_transcript` input variable in `PROMPT_TEMPLATE_FORMATTING_H`. This is where each lecture text goes.

Now, execute the following steps.

1. Use the `PROMPT_FORMATTING_S` template to create a system message; store it in the `prompt_formatting_s` variable.
2. Use the `PROMPT_TEMPLATE_FORMATTING_H` template to create a human message prompt template; store it in the `prompt_template_formatting_h` variable.
3. Create a chat prompt template (`chat_prompt_template_formatting`) from the system message and the human message prompt template.
3. Now that we have implemented the first component of the chain, it's time we move on to the chat object. Create a `ChatOpenAI` instance using the GPT-4o model, a seed of 365, and set the temperature to 0.
4. The final piece of the chain is to create the string output parser. Define an instance of the `StrOutputParser` class and call it `str_output_parser`.
5. Now, assemble the elements in a chain called `chain_formatting`. Use LCEL's pipe notation to pass the output from the chat prompt template as an input to the chat object and its corresponding output as an input to the output parser.
6. It's now time for us to invoke the chain. Use a method that can work on all lectures in the `string_list_split` list in parallel.

7. Print out the lectures to inspect the differences with the old script. If executed correctly, the text's formatting should be clearer, with precise punctuation and corrected mistranscriptions. Nice!

What's left is to substitute the lecture content of the `Document` objects in the `docs_list_md_split` list with these new texts. Override the `docs_list_md_split` so that it keeps `Documents` with improved texts.

## Split the Lectures with `TokenTextSplitter`

Okay, we now have a list of documents, each with a lecture from the Introduction to Tableau course. We've also improved the transcript with the help of the GPT-4o model. Still, glancing through the lectures, you'll find some are quite long. For this project phase, your task is to divide the lectures further, now adhering to a maximum token count.

Create a token text splitter that uses the "cl100k\_base" encoding, limits the chunk sizes to 500 tokens, and introduces a 50-tokens overlap between subsequent chunks. Use this object to split the documents in the `docs_list_md_split` list and store the result in a variable called `docs_list_tokens_split`.

To execute this task:

1. Create an instance of the `TokenTextSplitter` class with parameters `encoding_name`, `chunk_size`, and `chunk_overlap` set to the corresponding values.

2. Apply the `split_documents()` methods to the splitter, passing the `docs_list_md_split` as an argument.
3. Ensure the new document list is stored in the `docs_list_tokens_split` variable.

## Create Embeddings, Vector Store, and Retriever

Having split the text, we'll proceed to the next indexing phase: embedding the documents, storing them locally in a vector database, and establishing a vector store-backed retriever.

1. Create an OpenAI embedding instance called `embedding` that uses the "text-embedding-3-small" model.
2. Next, use the embedding object and the `docs_list_tokens_split` list to define a Chroma vector database called `vectorstore`. Store the embeddings in a local folder called "intro-to-tableau."
3. Finally, for the indexing part, use the Chroma vector store to create a `Runnable` retriever object that would later enter our LCEL Q&A chatbot chain. Apply the maximal marginal relevance search type, ensure the retriever fetches exactly two documents and set the lambda parameter of the MMR search to 0.7.

## Create Prompts and Prompt Templates for the Q&A Chatbot Chain

You've now reached the most intriguing part of the project—creating the components of the Q&A chatbot chain and, eventually, constructing the chain. In this section, your task is to create the prompts and prompt templates necessary for instructing the chatbot, providing the context, and feeding the students' questions.

Familiarize yourself with the strings `PROMPT_CREATING_QUESTION`, `PROMPT_RETRIEVING_S`, and `PROMPT_TEMPLATE_RETRIEVING_H`, which we'll explore further below. We've maintained consistent naming conventions, with "S" and "H" representing "system" and "human," respectively. Now, your task is to:

1. Create a prompt template using `PROMPT_CREATING_QUESTION`. Call it `prompt_creating_question`.
2. Create a system message using `PROMPT_RETRIEVING_S`. Call it `prompt_retrieving_s`.
3. Create a human message prompt template using `PROMPT_TEMPLATE_RETRIEVING_H`. Call it `prompt_template_retrieving_h`.
4. Create a chat prompt template combining the system message and the human message prompt template. Call it `chat_prompt_template_retrieving`.

If you need assistance creating these objects, refer to the similar instructions in Part 3: Create a Chain to Correct the Course Transcript.

## Create the First Version of the Q&A Chatbot Chain

All right, everyone! We're now ready to construct the first version of the Q&A chatbot chain. (The second and final version will feature an improvement in context formatting. We'll create this chain in the final two parts of the project.)

1. The line of code `result = chain_retrieving.invoke(...)` in the Jupyter Notebook file shows that a question comprises three elements:
  1. The corresponding lecture's name
  2. The question's title
  3. The question's body

The same line also tells us that the first component of the chain is `prompt_creating_question`.

As you can ensure, invoking the chain with this only component results in a `StringPromptValue` object.

2. Now, we must consider how we'll feed this result in the `{question}` input variable from `chat_prompt_template_retrieving`. The required format is a string and not a `StringPromptValue`.

Your task is to create a `RunnableLambda` function that extracts the text from the `StringPromptValue` object. Upon invoking this two-component chain, you should obtain the following string:

**Lecture:** Adding a custom calculation

**Title:** Why are we using SUM here? It's unclear to me.

**Body:** This question refers to calculating the GM%.

3. Create the third component of the chain so that upon invocation, you obtain the following dictionary:

```
{ 'context': [Document(...),  
              Document(...)],  
  'question': 'Lecture: Adding a custom calculation\n'  
              '"Title: Why are we using SUM here? It's unclear to  
me.\n"
```

```
'Body: This question refers to calculating the  
GM%.' }
```

4. Pipe the final three components of the chain so that when the chain is invoked, the chatbot's response is returned as a string.

## Create a Runnable Function to Format the Context

We're almost at the finish line. But there are still two more tasks lurking, and this one can be a bit tricky. 😊

Study the chat prompt template used for the chat model. You'll notice the `{context}` input variable is substituted with a list of two `Document` objects – a format that is not very clean or tidy. A better approach would be to format the context as follows:

```
Document 1  
Section Title: ...  
Lecture Title: ...  
Content: ...
```

-----

```
Document 1  
Section Title: ...  
Lecture Title: ...  
Content: ...
```

-----

Create a `Runnable` function named `format_context(...)` that accepts a dictionary formatted as

```
{ 'context': [Document(...),  
             Document(...)],  
  'question': ... }
```

and returns the dictionary with the `context` key as a string formatted as shown above.

## Stream the Response

Okay, you're about to cross the finish line! So far, you've created a helpful Q&A chatbot that answers students' questions throughout an entire 365 course and provides the resources it has used to construct the answer. Congratulations!

Your final task is to make the chatbot more user-friendly by streaming the tokens as soon as they're generated rather than having the chatbot complete the response before it's displayed to the user. Make the corresponding changes to the code and enjoy your fully functional chatbot application!

All right, everyone. I hope you've found this project interesting, valuable, and illuminating. I trust it allowed you to study LangChain's documentation, solidify your skills and knowledge, and learn more about this wonderful framework. You can enhance this chain with more functionalities and experiment with LangChain's integrations.

Good luck, and have fun!